

G-Lab — Report Generator Rewrite

Scope, 19 August 2026 · replacing the LLM-driven Friday report with deterministic Python

0. The finding that changes the order of work

122 of 169 analysis scripts read only `capture.db` — a rolling window of a few trading days — and none of them say so. They print "last 400d" or "full history" while scanning a fraction. This is not a reporting bug; it is a correctness bug in the research layer, and it silently invalidates any conclusion drawn over a period longer than the hot tier.

Three instances were found and fixed on 18–19 August: the run census, the walk-forward harness, and (after the census was fixed) the fix itself, which reintroduced the same hardcoded timeframe one file away. Two of the remaining 122 — `abs_veto_robustness.py` and `adaptive_exit_bt.py` — are invoked by the Friday report's own shadow section.

Therefore: the data-access layer is prerequisite work, not part of the rewrite. Rewriting the report generator on top of 122 scripts that each narrow their own dataset would produce a deterministic report that is confidently wrong — which is worse than an LLM-written one that is occasionally vague.

1. Current state, measured

Layer	Implementation	Lines
Capture, storage, lake, shadow sim, re-pricing, census, walk-forward, safety, reconciliation	Deterministic Python	75,036 (95%)
Router decision · hourly watch · scheduled reviews · Friday report · courtroom sims	Claude-dependent	3,208 (4%)

Only **7 files** invoke the CLI. The Friday report is **21 of 21 phases** Claude prompts, but **9 deterministic scripts already exist** (2,966 lines, zero Claude) doing the actual computation — the prompts largely orchestrate and narrate them.

2. Prerequisite — the data-access layer (do this first)

2.1 One reader, and it cannot silently narrow

```
gazbot7.lake.connect()          # already exists: V5 archive + Parquet lake + hot SQLite
|
▼
glab.data.bars(symbol, tf=None, since=None, until=None)
|   · tf=None -> choose the WIDEST-SPAN timeframe fine enough for the caller's need
|   · records what it scanned: source tier, timeframe, row count, span, gaps
|   · RAISES if the requested window exceeds what is held, never truncates
▼
every report component, every backtest, every study
```

Two rules learned the hard way, both non-obvious.

(a) Choose timeframe by COVERAGE, not by fineness. Preferring the finest bar picks 5-second data — which exists only for the weeks we captured ourselves — and discards nine months of backfilled minute bars to gain a resolution the measurement does not use. Selection must be widest-span among timeframes fine enough to resolve the question, with anything coarser refused rather than accepted.

(b) A label that misnames its source is worse than no label. The walk-forward printed "alphabot.db 1m" while reading the lake, because it echoed a default instead of what it used. Every component must emit the tier, timeframe, row count and span it actually scanned.

2.2 Migration of the 122

Step	Work	Effort
1	Build <code>glab.data</code> over the existing <code>lake.connect()</code> , with span reporting and refuse-on-overflow	2-3 days
2	A test that FAILS if any script imports <code>sqlite3</code> against <code>capture.db</code> directly — stops the population growing	half a day
3	Migrate the ~20 scripts the report actually calls	1 week
4	Migrate the long tail (102 exploratory scripts) opportunistically, or quarantine them as <code>research/legacy/</code>	ongoing

Step 2 matters more than it looks: without it, the next 122 arrive the same way the first did.

3. Phase-by-phase rewrite scope

21 phases. 7 greenfield + `movement3` are excluded (\$4). The remaining 13 fall into three tiers.

Tier A — templating over scripts that already exist

The computation is done; Claude is being used as a query planner and HTML formatter.

Phase	Existing deterministic script	Remaining work	Effort
day_rider	day_rider_lab.py (263 ln, already lake-aware)	HTML template	1 day
part2_shadow	abs_veto_robustness , adaptive_exit_bt , partial_shadow_watch , two_ratchet_shadow_watch (719 ln)	migrate all 4 to the lake, then template	3 days
part2_6_router	router_study.py (299 ln)	template + parse the router log	1-2 days
run_charts	run_charts.py (253 ln) — reads capture.db	migrate to lake, then template	1 day
assemble	friday_v7_build.py (1,053 ln) already builds HTML+PDF	drive it from a manifest instead of a prompt	1-2 days

Tier B — new Python, mechanical but real

No script exists; the logic is stated in the prompt and is deterministic once written down.

Phase	What it computes	Effort
part1_live	Live desk P&L by gate / period / regime, blotter, equity curve, exit mix	3-4 days
movement2_idle	Gates that did not fire, cross-referenced against census runs — needs the suppression journal	3 days
rehab	Gate ranking by the stated rehabilitation rules; the rules exist in prose and must be encoded	1 week
gf_report	Tabulates greenfield findings into the gates page — mechanical IF greenfield emits structured output (\$4)	2 days

Tier C — keep, drop, or narrow

Phase	Assessment
part25_musings	Drop. Commentary. A deterministic product should not ship opinions it cannot justify from a table.
proofread	Narrow to consistency checking , which IS deterministic: do the numbers in the prose match the tables, do totals reconcile, is any figure quoted with a different sample size elsewhere. That catches the errors that matter. Narrative review is not replaceable and is not needed.
rev2 , final	Replace with assertions. Their real job is "did every section actually build" — that is a manifest check, not a read-through. The 08-14 failure (rc=0 , artifact missing) is exactly what an assertion catches and a read-through did not.

4. Greenfield — the honest exception

The seven `gf_*` phases invoke **zero** Python scripts. They are wholly "here is a data contract, go find something". Three separable parts:

Part	Automatable?
Running a hypothesis sweep across a parameter surface	Yes — this is already what the backtest engine does
Applying the validation battery (leave-one-out, both-halves, placebo, neighbourhood)	Yes — deterministic, and partly built
Choosing which hypothesis space is worth searching, and recognising an artefact	No

Recommendation: do not remove it — make it optional and structured.

Greenfield stays LLM-driven behind a provider interface, and is required to emit **structured output** (candidate, mechanism, evidence, sample size, refutation condition) rather than prose. Then `gf_report` becomes templating, and a customer without an AI key still gets a complete instrument that MEASURES — just not one that PROPOSES.

5. Sequenced plan and effort

#	Work	Effort	Unblocks
1	Data-access layer + the test that prevents regression	3–4 days	everything; without it the rest is confidently wrong
2	Migrate the ~20 report-critical scripts to it	1 week	Tier A
3	Tier A — five phases to templates	1.5 weeks	~40% of the report
4	Tier B — four phases, new Python	2.5–3 weeks	~70% cumulative
5	Tier C — assertions + consistency checks; drop musings	1 week	the whole tail
6	Structured-output contract for greenfield	1 week	optional AI layer
	Total to a deterministic report with an optional AI research layer	7–8 weeks	

6. What this buys, commercially

- **No LLM dependency in the core.** A customer needs no AI key to get capture, storage, backtesting, census, walk-forward, validation and a full weekly report.
- **Removes an operational liability.** Session-based model credentials once silently no-op'd the router for 10.5 hours. Shipping that dependency to customers is not viable; here it becomes an optional add-on rather than a single point of failure.

- **Reproducibility.** A deterministic report run twice on the same data produces the same bytes. That is auditable in a way an LLM-written one is not — and it is what a professional buyer will ask for first.
- **Cost.** The per-run cost of the weekly report goes to approximately zero.
- **Honest positioning.** "Deterministic research instrument, optional AI analyst" is a stronger and more defensible claim than either "AI-powered" or "no AI".

The one thing not to promise. Hypothesis generation is the least automated link in the loop and the most tempting to overstate. The deterministic product measures, validates and reports. It does not invent the next question — and a scope that claims otherwise will be found out.

Prepared 19 August 2026 from direct inspection of the running system. Line counts, script counts and the 122/169 figure are measured, not estimated.